



INSTITUT INTERNATIONAL D'INGÉNIERIE DE L'EAU ET DE L'ENVIRONNEMENT (2iE)

Département Informatique, Intelligence Artificielle et Applications (IAA)

ECUE IAA511, Programmation Orientée Objet en Java, Semestre 5

Cahier des charges

Application de réservation de billets de transport

Bus, trains et vols au départ de Ouagadougou

Groupe Groupe 01

Membres SIONE Totta Yasser Soufiane

YELKUNI Tindwendé Franck Onel

NIKIEMA P. Hanifah

NADEMBEGA Ingrid Océane

Enseignant Dr Babacar LEYE

Version 3.0, version finale

Date 1er octobre 2026

Ce document décrit ce que le programme doit faire et comment nous vérifierons qu'il le fait. Il sert de référence commune pour le code, les tests et le rapport.

Table des matières

1	Présentation du projet	1
1.1	Contexte	1
1.2	Objectifs	1
1.3	Périmètre	1
2	Vocabulaire et acteurs	2
2.1	Acteurs	2
2.2	Glossaire	2
3	Besoins fonctionnels	3
4	Règles de gestion	4
4.1	Cycle de vie d'un ticket	5
5	Contraintes techniques et qualité	5
6	Exigences liées au cours	6
7	Architecture attendue	7
8	Interface utilisateur	7
9	Scénarios d'acceptation	8
10	Stratégie de test et critères de recette	9
11	Livrables	10
12	Organisation du travail	10
12.1	Répartition	10
12.2	Étapes	11
13	Risques et solutions	11
14	Évolutions possibles	11

Liste des tableaux

Tableau 1	Acteurs du système et leur rôle	2
Tableau 2	Glossaire des termes du projet	2
Tableau 3	Besoins fonctionnels	3
Tableau 4	Règles de gestion	4
Tableau 5	Transitions possibles du statut d'un ticket	5
Tableau 6	Contraintes techniques et exigences de qualité	5
Tableau 7	Notions du cours et application dans le projet	6
Tableau 8	Packages du projet et leur contenu	7
Tableau 9	Options du menu interactif	7
Tableau 10	Scénarios d'acceptation	8
Tableau 11	Livrables du projet	10
Tableau 12	Répartition des responsabilités	10
Tableau 13	Risques identifiés et solutions prévues	11

1 Présentation du projet

1.1 Contexte

Une agence de transport souhaite suivre ses trajets et ses voyageurs. Elle propose trois façons de voyager : le bus, le train et l'avion. Un voyageur réserve une place sur un trajet, ce qui crée un ticket. Tant que le délai le permet, le voyageur peut annuler ce ticket. Quand l'heure du départ est atteinte sans annulation, le ticket est validé automatiquement et considéré comme payé.

Les délais d'annulation ne sont pas les mêmes selon le moyen de transport. Pour un bus, l'annulation reste possible jusqu'à 24 heures avant le départ. Pour un train, elle doit se faire au moins 48 heures avant. Pour un vol, elle doit se faire au moins 72 heures avant, car les compagnies aériennes bloquent leurs places plus tôt.

Lorsqu'un trajet est complet, le voyageur n'est pas simplement refusé : il est placé dans une file d'attente et reçoit automatiquement la première place libérée par une annulation. Dans cette version finale, l'agence peut aussi être sauvegardée dans un fichier puis rechargée, et un menu interactif permet d'utiliser le programme sans écrire de code.

1.2 Objectifs

Le projet poursuit cinq objectifs.

1. Fournir un programme Java fiable qui gère les réservations, les annulations, les validations et les files d'attente.
2. Appliquer dans le code les notions vues en cours : interfaces, classe abstraite, polymorphisme, généricité, collections, contrat `equals()` et `hashCode()`, exceptions personnalisées, entrées et sorties, sérialisation et tests unitaires.
3. Séparer clairement le modèle, la logique métier et l'interface utilisateur : seules les classes du package d'interface écrivent à l'écran.
4. Garantir qu'une opération refusée ne laisse aucune trace : ni statut modifié, ni place perdue, ni fichier partiel.
5. Prouver chaque comportement par un test automatique, y compris les cas d'erreur.

1.3 Périmètre

Ce que le programme fait dans cette version

- la gestion des voyageurs, des trajets (bus, train et vol) et des tickets ;
- la réservation, l'annulation, la validation au départ et la file d'attente ;
- les recherches, les tris et les statistiques ;
- la sauvegarde et le rechargement de l'agence dans un fichier ;
- une démonstration complète et un menu interactif en console.

Ce que le programme ne fait pas

- pas de vraies dates ni d'horloge : le délai avant le départ est un nombre d'heures donné à la création du trajet ;
- pas de paiement en ligne ni de base de données : les données restent en mémoire, sauf si on les sauvegarde dans un fichier ;
- pas d'interface graphique : l'interface est une application console ;
- pas de choix de siège : seul le nombre de places disponibles est suivi.

2 Vocabulaire et acteurs

2.1 Acteurs

Tableau 1: Acteurs du système et leur rôle

Acteur	Rôle
Voyageur	Réserve, annule et consulte ses tickets.
Agent de l'agence	Enregistre les voyageurs, ajoute les trajets, consulte les statistiques, sauvegarde et recharge l'agence.
Système	Valide automatiquement les tickets au départ et attribue les places libérées aux voyageurs en attente.

2.2 Glossaire

Tableau 2: Glossaire des termes du projet

Terme	Définition
Trajet	Un déplacement d'une ville de départ vers une ville d'arrivée, à une date et une heure, avec un nombre de places.
Ticket	Le lien entre un voyageur et un trajet. Il porte un prix en FCFA et un statut.
Statut	L'état d'un ticket : RESERVE, PAYE, ANNULE, REFUSE ou EN_ATTENTE.
Délai d'annulation	Nombre d'heures avant le départ au delà duquel l'annulation est refusée : 24 pour un bus, 48 pour un train, 72 pour un vol.
File d'attente	Liste des tickets en attente d'une place sur un trajet complet, servie dans l'ordre d'arrivée.
Atomicité	Une opération réussit complètement ou laisse tout dans son état initial.
Sérialisation	Transformation d'un objet en suite d'octets pour l'écrire dans un fichier, puis reconstruction de l'objet depuis ce fichier.

3 Besoins fonctionnels

Chaque besoin porte un identifiant repris dans le rapport et dans les tests. La priorité vaut 1 pour indispensable et 2 pour important.

Tableau 3: Besoins fonctionnels

Id	Description	Priorité
BF1	Enregistrer un voyageur avec un numéro, un nom non vide et un email valide. Le numéro et l'email doivent être uniques dans l'agence.	1
BF2	Ajouter un trajet en bus, en train ou en vol avec un numéro unique, deux villes différentes, un nombre de places positif ou nul et un délai avant départ positif ou nul.	1
BF3	Lister les trajets disponibles (non complets), du plus proche au plus lointain.	2
BF4	Rechercher les trajets d'une ville de départ vers une ville d'arrivée, sans tenir compte des majuscules.	2
BF5	Réserver une place : un ticket RESERVE est créé et une place est retirée du trajet.	1
BF6	Refuser la réservation si le voyageur possède déjà un ticket actif sur le même trajet.	1
BF7	Si le trajet est complet, placer le ticket en file d'attente (EN_ATTENTE) sans retirer de place.	1
BF8	Annuler un ticket RESERVE tant que le délai d'annulation est respecté. La place est rendue.	1
BF9	Refuser l'annulation hors délai ou pour un ticket qui n'est pas annulable, sans modifier aucun état.	1
BF10	Attribuer automatiquement la place libérée au premier ticket de la file d'attente du trajet et signaler les tickets promus.	1
BF11	Permettre d'annuler un ticket en file d'attente : il quitte la file et aucune place n'est libérée.	2
BF12	Valider automatiquement au départ les tickets RESERVE d'un trajet : ils passent PAYE.	1
BF13	Consulter les tickets d'un voyageur, les voyageurs d'un trajet et les voyageurs présents sur deux trajets à la fois.	2
BF14	Calculer le nombre de tickets par statut, le chiffre d'affaires (tickets payés) et la liste des villes desservies.	2
BF15	Classer les voyageurs par ordre alphabétique de leur nom.	2
BF16	Signaler chaque erreur métier par une exception personnalisée dont le nom et le message sont explicites.	1
BF17	Donner la position d'un ticket dans la file d'attente de son trajet.	2

Id	Description	Priorité
BF18	Sauvegarder l'agence complète dans un fichier et la recharger plus tard, sans perdre les tickets, les statuts ni les files d'attente.	2
BF19	Proposer un menu interactif en console qui permet d'utiliser toutes les fonctions précédentes.	2
BF20	Ne jamais écrire en console depuis le modèle ou le service : les résultats sont renvoyés et c'est l'interface qui les affiche.	1

4 Règles de gestion

Tableau 4: Règles de gestion

Id	Règle
RG1	Un bus peut être annulé si le départ est dans plus de 24 heures, un train dans plus de 48 heures, un vol dans plus de 72 heures. À exactement 24 heures pour un bus, l'annulation est refusée.
RG2	Le nombre de places disponibles d'un trajet n'est jamais négatif.
RG3	Le prix d'un ticket est un nombre positif ou nul, exprimé en FCFA.
RG4	Le nom d'un voyageur ne peut pas être vide et son email doit contenir le caractère @. Deux voyageurs ne peuvent pas partager le même email, majuscules ignorées.
RG5	Les numéros de voyageur et de trajet sont uniques. Les tickets sont numérotés automatiquement par l'agence à partir de 1.
RG6	Un ticket créé directement sur un trajet complet naît REFUSE. Quand il passe par l'agence, il devient EN_ATTENTE.
RG7	Seul un ticket RESERVE peut passer PAYE lors de la validation au départ.
RG8	Un ticket ANNULE, PAYE ou REFUSE ne peut plus être annulé.
RG9	La file d'attente fonctionne dans l'ordre d'arrivée : le premier arrivé est le premier servi.
RG10	Un ticket ANNULE ou REFUSE n'est plus actif : le voyageur peut réserver à nouveau le même trajet.
RG11	Le chiffre d'affaires ne compte que les tickets PAYE.
RG12	Une opération refusée ne modifie ni le statut d'un ticket, ni le nombre de places, ni les collections de l'agence.
RG13	Une sauvegarde est d'abord écrite dans un fichier temporaire, puis déplacée vers sa destination. Si elle échoue, l'ancien fichier reste intact et aucun fichier partiel n'est laissé.
RG14	Un fichier qui n'est pas une sauvegarde valide est refusé par une Sauvegarde Exception, et l'agence en cours n'est pas modifiée.

Id	Règle
RG15	Dans le menu, une saisie invalide (du texte à la place d'un nombre, un email sans arobase) est signalée sans arrêter le menu et sans rien modifier.

4.1 Cycle de vie d'un ticket

Tableau 5: Transitions possibles du statut d'un ticket

Départ	Événement	Arrivée	Condition
Création	Réservation sur un trajet avec une place libre	RESERVE	Trajet non complet
Création	Réservation directe sur un trajet complet	REFUSE	Trajet complet
REFUSE	Mise en file par l'agence	EN_ATTENTE	Passage par l'agence
EN_ATTENTE	Une place se libère	RESERVE	Premier de la file
EN_ATTENTE	Annulation par le voyageur	ANNULE	Aucune
RESERVE	Annulation par le voyageur	ANNULE	Délai respecté
RESERVE	Validation au départ	PAYE	Aucune

Note. Les statuts PAYE et ANNULE sont des états finaux.

5 Contraintes techniques et qualité

Tableau 6: Contraintes techniques et exigences de qualité

Contrainte	Exigence
Langage	Java 11 ou plus. Le projet a été vérifié avec Java 21.
Tests	JUnit 5 pour les tests unitaires, plus un programme de scénarios sans dépendance externe.
Interface	Application console. Les textes affichés évitent les caractères spéciaux pour rester lisibles sur tous les terminaux.
Données	En mémoire, avec sauvegarde facultative dans un fichier.
Code	Noms de classes, de méthodes et de variables en français, explicites. Aucun commentaire n'est nécessaire dans le code : les noms suffisent à le lire.
Robustesse	Toute donnée invalide est refusée par une exception, jamais ignorée.
Lisibilité	Une classe, une responsabilité. Les méthodes restent courtes.

Contrainte	Exigence
Portabilité	Le projet se compile avec un simple <code>javac</code> ou dans IntelliJ IDEA. Des scripts sont fournis pour Linux, macOS et Windows.

6 Exigences liées au cours

Le professeur demande d'utiliser dans le projet les notions et les structures du cours. Le tableau 7 indique où chacune est appliquée.

Tableau 7: Notions du cours et application dans le projet

Notion du cours	Application dans le projet
Interface	<code>Affichable</code> et <code>Occupable</code> . <code>Occupable</code> remplace l'ancienne interface <code>Reservable</code> .
Classe abstraite et héritage	<code>Trajet</code> , avec la méthode abstraite <code>delaiAnnulationHeures()</code> . <code>TrajetBus</code> , <code>TrajetVol</code> et <code>TrajetTrain</code> en héritent.
Polymorphisme	Le ticket interroge <code>delaiAnnulationHeures()</code> sans jamais tester le type réel du trajet. Un tableau d' <code>Affichable</code> est parcouru dans une seule boucle.
Généricité	La classe <code>Depot<T></code> range des voyageurs ou des trajets selon son type.
Listes	<code>ArrayList</code> pour les tickets de l'agence et les résultats de recherche.
Ensembles	<code>HashSet</code> pour les emails et les voyageurs d'un trajet, <code>TreeSet</code> pour les villes desservies.
Mappes	<code>TreeMap</code> dans <code>Depot<T></code> et pour compter les tickets par statut, <code>HashMap</code> pour les files d'attente.
File	Queue implémentée par <code>LinkedList</code> pour la file d'attente de chaque trajet.
Contrat <code>equals()</code> et <code>hashCode()</code>	Redéfinis ensemble dans <code>Voyageur</code> , <code>Trajet</code> et <code>Ticket</code> .
Comparable et <code>Comparator</code>	<code>Trajet</code> et <code>Voyageur</code> sont comparables. Comparateur <code>VoyageurParNom</code> donne un second ordre.
Parcours et méthodes utiles	Boucles <code>for-each</code> , <code>retainAll()</code> , <code>Collections.sort()</code> .
Exceptions vérifiées	Hierarchie de six exceptions qui héritent de <code>BilletterieException</code> .
Exceptions non vérifiées	<code>IllegalArgumentException</code> pour les erreurs de programmation (nom vide, prix négatif).
<code>try</code> , <code>catch</code> , <code>finally</code>	Utilisés dans <code>Main</code> , <code>MenuConsole</code> et les tests.

Notion du cours	Application dans le projet
Propagation	Les méthodes de l'agence laissent remonter les exceptions jusqu'à l'interface.
Entrées et sorties	Scanner pour le menu, Files et flux d'objets pour la sauvegarde.
Sérialisation	Les classes du modèle et de l'agence implémentent <code>Serializable</code> .
Atomicité	Validation avant toute modification, et sauvegarde par fichier temporaire.
Tests unitaires	Huit classes JUnit 5 et un programme de scénarios.

7 Architecture attendue

Tableau 8: Packages du projet et leur contenu

Package	Contenu et rôle
<code>billetterie.contrats</code>	Les interfaces <code>Affichable</code> et <code>Occupable</code> .
<code>billetterie.modele</code>	Le modèle : <code>Voyageur</code> , <code>Trajet</code> , <code>TrajetBus</code> , <code>TrajetVol</code> , <code>TrajetTrain</code> , <code>Ticket</code> , <code>StatutTicket</code> .
<code>billetterie.exceptions</code>	<code>BilletterieException</code> et ses six exceptions filles.
<code>billetterie.service</code>	La logique métier : <code>Agence</code> , <code>Depot<T></code> , <code>CompareurVoyageurParNom</code> , <code>Sauvegarde</code> .
<code>billetterie.ui</code>	L'interface console : <code>Main</code> (démonstration) et <code>MenuConsole</code> (menu interactif).
<code>billetterie.tests</code>	Le programme de scénarios <code>TestsScenarios</code> .

Le dossier `test` contient les classes JUnit 5, dans les mêmes packages que le code testé.

8 Interface utilisateur

Le programme se lance de deux façons. Sans argument, `Main` déroule une démonstration complète. Avec l'argument `menu`, il ouvre le menu interactif décrit dans le tableau 9.

Tableau 9: Options du menu interactif

Choix	Action
1	Lister les trajets disponibles, du plus proche au plus lointain.
2	Rechercher un trajet par ville de départ et ville d'arrivée.

Choix	Action
3	Enregistrer un voyageur (numéro, nom, email).
4	Réserver une place (voyageur, trajet, prix). La position dans la file est annoncée si le trajet est complet.
5	Annuler un ticket. Les tickets promus depuis la file sont signalés.
6	Valider le départ d'un trajet.
7	Voir les tickets d'un voyageur.
8	Afficher les statistiques de l'agence.
9	Sauvegarder l'agence dans un fichier.
10	Charger une agence depuis un fichier.
0	Quitter le menu.

9 Scénarios d'acceptation

Le programme est accepté quand tous les scénarios du tableau 10 donnent le résultat attendu.

Tableau 10: Scénarios d'acceptation

Id	Scénario	Résultat attendu
SA1	Réserver une place sur un bus de 2 places.	Ticket RESERVE, 1 place restante.
SA2	Réserver sur un trajet de 0 place avec la classe Ticket.	Ticket REFUSE, places inchangées.
SA3	Annuler un ticket de bus avec un départ dans plus de 24 heures.	Ticket ANNULE, place rendue.
SA4	Annuler un ticket de vol avec un départ dans moins de 72 heures.	Exception, statut et places inchangés.
SA5	Annuler un ticket de train avec un départ dans moins de 48 heures.	Exception, statut et places inchangés.
SA6	Annuler deux fois le même ticket.	Seconde annulation refusée, une seule place rendue.
SA7	Réserver deux fois le même trajet avec le même voyageur.	ReservationInvalideException, aucun ticket créé.
SA8	Enregistrer un voyageur avec un email déjà pris.	DoublonException.
SA9	Réserver avec un voyageur ou un trajet inconnu.	ElementIntrouvableException.
SA10	Quatre voyageurs pour un bus de 3 places.	Trois RESERVE, un EN_ATTENTE.

Id	Scénario	Résultat attendu
SA11	Annuler un ticket alors qu'un voyageur attend.	Le premier de la file passe RESERVE.
SA12	Annuler un ticket en file d'attente.	Ticket ANNULE, file réduite, aucune place rendue.
SA13	Valider le départ d'un trajet.	Seuls les tickets RESERVE passent PAYE.
SA14	Calculer le chiffre d'affaires.	Somme des prix des tickets PAYE uniquement.
SA15	Chercher les voyageurs présents sur deux trajets.	Intersection correcte des deux ensembles.
SA16	Créer un voyageur sans nom, un trajet aux villes identiques, un prix négatif.	<code>IllegalArgumentException</code> .
SA17	Sauvegarder l'agence puis la recharger.	Mêmes tickets, mêmes statuts, même file d'attente.
SA18	Charger un fichier absent ou invalide.	<code>SauvegardeException</code> , agence en cours inchangée.
SA19	Sauvegarder dans un dossier inexistant.	<code>SauvegardeException</code> , aucun fichier créé.
SA20	Saisir du texte à la place d'un nombre dans le menu.	Message d'erreur, menu toujours actif, rien de modifié.

10 Stratégie de test et critères de recette

Les tests sont écrits à deux niveaux.

1. Le programme `TestsScenarios` exécute 22 scénarios et 182 vérifications, et affiche [OK] ou [ECHEC] pour chacune, avec un bilan final. Il ne dépend d'aucune bibliothèque.
2. Huit classes JUnit 5 contiennent 109 tests : `VoyageurTest`, `TrajetTest`, `TrajetTrainTest`, `TicketTest`, `DepotTest`, `AgenceTest`, `SauvegardeTest` et `MenuConsoleTest`.

Chaque test d'exception vérifie deux choses : le type de l'exception levée et l'absence d'effet de bord observable, c'est-à-dire l'état avant et après l'opération.

Critères de recette

- le projet compile sans erreur ni avertissement ;
- `TestsScenarios` affiche zéro échec ;
- tous les tests JUnit réussissent ;
- `Main` se déroule de bout en bout sans exception non gérée, et le menu répond à toutes les

options ;

- les livrables du tableau 11 sont présents.

11 Livrables

Tableau 11: Livrables du projet

Livable	Contenu
Dossier <code>src</code>	Le code source du programme et le programme de scénarios.
Dossier <code>test</code>	Les tests unitaires JUnit 5.
Dossier <code>scripts</code>	Les scripts de compilation et de lancement pour Linux, macOS et Windows.
<code>README.md</code>	Présentation, structure, lancement et choix de conception.
<code>docs/cahier-des-charges.pdf</code>	Le présent document.
<code>docs/rapport-projet.pdf</code>	Le rapport de projet.
<code>docs/diagramme-classes.pdf</code>	Le diagramme de classes en trois pages.
Sources <code>.tex</code>	Les fichiers LaTeX de chaque document PDF.

12 Organisation du travail

12.1 Répartition

Tableau 12: Répartition des responsabilités

Membre	Responsabilités principales
SIONE Totta Yasser Soufiane	Trajet, TrajetBus, TrajetVol, TrajetTrain, Occupable, Depot<T>.
YELKUNI Tindwendé Franck Onel	Ticket, Agence et la file d'attente.
NIKIEMA P. Hanifah	Voyageur, les exceptions, Sauvegarde et le diagramme de classes.
NADEMBEGA Ingrid Océane	Main, MenuConsole, TestsScenarios et les tests JUnit.

La rédaction du cahier des charges, du rapport et du README est partagée. Chaque membre relit le travail des autres avant le rendu.

12.2 Étapes

1. Reprendre la version précédente et lister les retours du professeur.
2. Renommer l'interface, ajouter les packages et les nouvelles classes.
3. Écrire les tests en même temps que chaque classe.
4. Ajouter le train, la sauvegarde et le menu interactif.
5. Mettre à jour le diagramme de classes, le README, le cahier des charges et le rapport.
6. Relire, compiler une dernière fois, lancer tous les tests et préparer l'archive.

13 Risques et solutions

Tableau 13: Risques identifiés et solutions prévues

Risque	Solution prévue
Un refus modifie l'état par erreur	Valider avant de modifier, puis tester l'état avant et après.
Doublons non détectés dans un <code>HashSet</code>	Redéfinir <code>equals()</code> et <code>hashCode()</code> ensemble, avec un test dédié.
Ordre d'affichage imprévisible	Ne jamais compter sur l'ordre d'un <code>HashSet</code> : trier avant d'afficher ou utiliser un <code>TreeSet</code> .
Sauvegarde interrompue qui détruit l'ancien fichier	Écrire dans un fichier temporaire puis le déplacer.
Fichier de sauvegarde invalide ou modifié	Capturer toutes les erreurs de lecture dans une <code>SauvegardeException</code> .
Saisie invalide dans le menu	Convertir les saisies avec contrôle et signaler l'erreur sans arrêter le menu.
Fichier rendu incomplet	Contrôler le contenu de l'archive avant de l'envoyer.

14 Évolutions possibles

Le programme peut évoluer sans casser l'existant, grâce aux interfaces et à la classe abstraite. L'ajout du train en est la preuve : il n'a demandé qu'une classe fille de `Trajet`, sans toucher à `Ticket`. Nous pourrions ajouter un bateau de la même façon, utiliser de vraies dates avec `LocalDateTime`, proposer une interface graphique et gérer des sièges numérotés.